

Atividade Prática 1 — Classificador Bayesiano

Etapa 3 — Implementação do Classificador Naive Bayes em SQL

Autor	Marcus Vinícius Santos de Almeida
Disciplina	Mineração de Dados
Data de entrega	04/09/2026
Banco de dados	DuckDB — arquivo dados/classificador.db
Artefatos	sql/classificador_naive_bayes.sql · sql/rodar_classificador.py
Entrada de treino	dados/etapa2_dados_treinamento.csv → tabela dados_treinamento (1.000.000 de linhas)

Este relatório explica a **arquitetura** do código para apoiar a defesa oral — não repete o SQL linha a linha (isso está nos comentários do próprio arquivo). A lógica SQL é essencialmente a mesma da versão anterior deste relatório (mesmas 5 views, mesma fórmula de Laplace, mesma normalização em log); o que mudou é (a) a migração do banco de SQLite para DuckDB e (b) os dados de entrada, agora N = 1.000.000 registros (ver CLAUDE.md, "Migração para DuckDB e N = 1.000.000 registros") — os números de exemplo abaixo foram recalculados sobre a massa atual.

1. Por que DuckDB

SQLite é um motor **transacional**, otimizado para muitas escritas pequenas e intercaladas. Este projeto tem o padrão oposto: os dados são escritos **uma vez** (a importação do CSV) e depois só sofrem **agregação** repetida (`GROUP BY`, `JOIN` nas views) — exatamente o padrão para o qual um motor **analítico e colunar** como o DuckDB foi desenhado. Com N = 1.000.000 de registros (e `treino_longo` chegando a 6.000.000 de linhas — §2), esse descompasso deixou de ser hipotético: DuckDB escala para esse volume sem esforço extra, enquanto SQLite começaria a sentir o padrão de acesso errado para seu motor.

A troca **mantém** as vantagens do SQLite que já estavam documentadas na versão anterior deste relatório:

- **Roda sem servidor.** É um único arquivo (`classificador.db`); não há processo a subir, porta a abrir nem usuário/senha. Basta `import duckdb`.
- **Fácil de testar no ambiente.** O script de teste (re)cria o banco do zero, importa o CSV, roda o classificador e confere o resultado em segundos — sem servidor nem dependências externas de infraestrutura.
- **SQL padrão suficiente.** O classificador usa apenas `CREATE VIEW`, `JOIN`, `GROUP BY`, CTEs (`WITH`), `UNION ALL`, `CASE` e as funções `LN/EXP` — tudo disponível nativamente no DuckDB, como já era no SQLite.

E **ganha** três coisas na troca:

1. **LN/EXP nativos, sem fallback.** O DuckDB traz essas funções matemáticas nativamente; diferente do SQLite, não é preciso nenhum código de contingência em Python para garanti-las (ver §4, mais abaixo — a seção equivalente da versão anterior deste relatório foi removida por não ser mais necessária).
2. **Importação de CSV mais direta.** `read_csv_auto('arquivo.csv')` faz a leitura, inferência de tipos e carga em uma única instrução SQL, em vez do laço manual de `INSERT` linha a linha que o SQLite exigia (§2).
3. **Interface visual nativa (`duckdb -ui`).** Útil para inspecionar as views ao vivo durante a apresentação oral, sem precisar de ferramenta externa (ver README).

Custo assumido: assim como o SQLite, o DuckDB não tem `PIVOT` nativo para essa finalidade — o "unpivot" continua feito à mão com `UNION ALL` (Seção 3.2), o que na verdade deixa a lógica mais explícita.

2. Fluxo de dados (do CSV ao veredito)

```
CSV da Etapa 2
| importa (rodar_classificador.py, via read_csv_auto)
v
dados_treinamento ..... 1.000.000 módulos, formato largo
|
+--> (a) priors ..... P(SIM), P(NAO)
|
+--> (b) treino_longo .... 6.000.000 de linhas, formato longo
    |
    v
    (c) verossimilhanças P(categoria | classe) + Laplace
    |
    caso_teste | (6 linhas: o modulo novo a classificar)
    \         |
    v         v
    (d) score_log ..... ln P(classe) + soma de ln P(cat|classe)
    |
    v
    (e) classificar_modulo P(SIM)%, P(NAO)%, recomendacao
```

3. As views, uma a uma

3.1 priors — probabilidade a priori P(classe)

Conta quantos módulos de treino são **SIM** e quantos são **NAO** e divide pelo total. É o palpite inicial, antes de olhar qualquer feature.

Resultado com a massa da Etapa 2: **$P(\text{NAO}) = 650.000/1.000.000 = 0,6500$** e **$P(\text{SIM}) = 350.000/1.000.000 = 0,3500$** . A proporção de classes é equivalente à da versão anterior do dataset (então 34,7 % / 65,3 %) — o gerador fixa **`PROPORCAO_DEFEITO = 0,35`** via seleção top-k, independentemente de como as 6 features são sorteadas (Etapa 2, §1.4); com N maior, o arredondamento de **`round(N × 0,35)`** bate exatamente em 35,0 %.

3.2 treino_longo — "unpivot" das 6 categorias

`dados_treinamento` tem uma coluna de categoria por feature (**`complexidade_cat`**, **`loc_cat`**, ...). Para calcular a verossimilhança de todas as features com **uma** consulta em vez de seis quase iguais, empilhamos os seis recortes com **`UNION ALL`**, chegando ao formato longo:

`(modulo_id, defeito, feature, categoria)` — 1.000.000 de módulos × 6 features = **6.000.000 de linhas**.

O nome curto de cada feature (**`complexidade`**, **`loc`**, **`n_autores`**, **`churn`**, **`n_imports`**, **`cobertura`**) passa a ser a chave usada no resto do pipeline e também nos casos de teste.

3.3 verossimilhanças — P(categoria | classe) com suavização de Laplace

Para cada combinação (feature, categoria, classe), a probabilidade condicional é

$P(\text{feature} = \text{categoria} \mid \text{classe}) = (\text{contagem} + 1) / (\text{total_da_classe} + k)$, com **$k = 3$** (as três categorias possíveis: **`baixo`**, **`medio`**, **`alto`**), igual para as 6 features.

Por que o "+1" e o "+k" (Laplace): sem eles, se uma categoria nunca aparece junto com uma classe no treino, a contagem é 0 e a probabilidade condicional é 0. Como o Naive Bayes combina as features multiplicando (ou, aqui, somando os logs), **um único zero anula a classe inteira** — o modelo fica "certo demais" por causa de uma célula vazia que é só falta de dados. O Laplace adiciona meia folga a cada categoria: nenhuma probabilidade é exatamente 0 nem exatamente 1, e o efeito sobre as categorias bem povoadas é desprezível. Como bônus, as três probabilidades de cada (feature, classe) passam a **somar exatamente 1**.

A view gera primeiro **todas** as $6 \times 3 \times 2 = 36$ combinações possíveis (CROSS JOIN de features, categorias e classes) e só então faz **LEFT JOIN** com as contagens observadas — assim as combinações ausentes no treino entram com contagem 0 e recebem o Laplace.

Exemplo concreto: `complexidade = alto` na classe `SIM` aparece 83.046 vezes; $P = (83.046 + 1) / (350.000 + 3) = 83.047/350.003 \approx 0,2373$.

3.4 `score_log` — somar logaritmos em vez de multiplicar

Para cada classe, o Naive Bayes calcula

```
score(classe) = P(classe) × P(cat1|classe) × ... × P(cat6|classe).
```

Por que log: cada fator é um número menor que 1 (aqui entre ~0,02 e ~0,7). Multiplicar 7 números pequenos dá um valor minúsculo; com muitas features isso chega a **underflow** — o `float` vira 0 e a informação some. Tomando logaritmo, o produto vira **soma**:

```
log score(classe) = ln P(classe) + Σi ln P(cati | classe)
```

Somar sete números da ordem de -1 a -4 é numericamente seguro. E como `ln` é crescente, a classe de maior log-score é a de maior probabilidade.

A view junta as 6 linhas do caso (`caso_teste`) com as verossimilhanças das **duas** classes e agrupa por (caso, classe), produzindo 2 linhas por caso. A coluna `n_features` deve dar 6 — se vier menor, alguma categoria do caso não casou (nome de feature errado ou categoria fora de `baixo/medio/alto`).

3.5 `classificar_modulo` — normalização e recomendação

Converte os dois log-scores de volta para probabilidade entre 0 e 100 %:

```
P(SIM) = exp(log_sim) / ( exp(log_sim) + exp(log_ao) )
```

O denominador comum `P(caso)` desaparece nessa divisão; reexponenciar e dividir pela soma reintroduz a normalização. Antes do `exp()` subtraímos `log_max = max(log_sim, log_ao)` dos dois expoentes — isso **não muda o resultado** (o fator sai igual no numerador e no denominador), só evita passar argumentos muito negativos ao `exp()`. É a forma numericamente correta da conta.

A recomendação textual sai de um `CASE`:

Condição	Recomendação
<code>P(SIM) > 50 %</code>	<code>ALTO RISCO – recomenda-se revisão de código e testes adicionais</code>
<code>caso contrário</code>	<code>BAIXO RISCO – pode seguir o fluxo normal</code>

4. Funções `LN` e `EXP` no DuckDB

O DuckDB traz `ln()` e `exp()` **nativas**, sem exigir nenhuma flag de build — confirmado com `SELECT ln(1), exp(0)`. Diferente da versão anterior deste relatório (SQLite), não é preciso nenhum código de contingência em Python (`connection.create_function`) para garantir essas funções: o `.sql` roda direto em qualquer instalação padrão do DuckDB. Essa seção existia na versão anterior por causa da possibilidade — remota, mas real no SQLite — de um build sem `SQLITE_ENABLE_MATH_FUNCTIONS`; no DuckDB esse cenário não existe, então a seção (e o código correspondente em `rodar_classificador.py`) foi removida.

5. Teste de ponta a ponta

`sql/rodar_classificador.py` recria o banco, roda o SQL e classifica **dois casos de exemplo mínimos** — apenas um *smoke test* para provar que o pipeline funciona. Os 5+ casos formais e a análise crítica são da Etapa 4.

Caso	Perfil (6 categorias)	P(SIM)	P(NAO)	Recomendação
exemplo_alto_risco	complexidade alto, loc baixo, autores alto, churn alto, imports alto, cobertura baixo	96,49 %	3,51 %	ALTO RISCO
exemplo_baixo_risco	complexidade baixo, loc medio, autores baixo, churn baixo, imports baixo, cobertura alto	4,41 %	95,59 %	BAIXO RISCO

Verificações automáticas do script (falha ⇒ código de saída 1):

- as duas probabilidades **somam 100,00 %** nos dois casos;
- **6/6 features** casaram com a tabela de verossimilhanças;
- o veredito bate com a intuição do domínio: arquivo pequeno, denso, muito mexido e mal coberto → alto risco; arquivo médio, simples, estável e bem coberto → baixo risco.

Log-scores obtidos (mostram o log em ação, longe de qualquer underflow): exemplo_alto_risco → ln-score SIM -8,8923 vs. NAO -12,2072; exemplo_baixo_risco → ln-score SIM -9,9617 vs. NAO -6,8849.

6. Como executar

```
python3 sql/rodar_classificador.py
```

O script é idempotente: recria dados_treinamento a partir do CSV e derruba/cria todas as views (DROP ... IF EXISTS), então pode ser rodado quantas vezes for preciso. Para inspecionar o modelo manualmente: duckdb dados/classificador.db "SELECT * FROM verossimilhanças;" — ou, para uma interface visual local (útil na apresentação): duckdb -ui dados/classificador.db.

Nota sobre o uso de IA

O SQL e o runner foram escritos em diálogo com uma IA generativa; cada view foi revisada e testada (contagens conferidas à mão, soma das verossimilhanças = 1, probabilidades finais somando 100 %). Todo o código é comentado bloco a bloco e é defensável oralmente.